@CLC_____ Number

UDC_____ Available for reference Yes No ☐ ☐

Undergraduate Thesis

Thesis Title：

Sign Language Recognition via Supervised Deep Learning

Student Name：Tha Davong
Student ID：11910227
Department： Computer Science and Engineering
Program： Computer Science and Technology
Thesis Advisor：于仕琪

Date: 2/June/2023

Letter of Commitment for Integrity

1. I solemnly promise that the paper presented comes from my independent research work under my supervisor's supervision. All statistics and images are real and reliable.
2. Except for the annotated reference, the paper contents no other published work or achievement by person or group. All people making important contributions to the study of the paper have been indicated clearly in the paper.
3. I promise that I did not plagiarize other people's research achievement or forge related data in the process of designing topic and research content.
4. If there is violation of any intellectual property right, I will take legal responsibility myself.

Signature: 

Date: 2/June/2023

Abstract

This paper attempt to explore a lightweight approach for automating sign language recognition using deep learning. Since there are two major categories of sign language: static sign language and dynamic sign language, we shall dedicate a separate section for each of them. For each category, a lightweight recognition pipeline will be proposed. Although the implementation detail of each pipeline varies greatly, they both follow the same major process: data augmentation, feature extraction, preprocessing, and feature recognition. To access the performance of each pipeline, three datasets (ASL alphabet, LSA64, and SM_ASL) are used to train the model on. The first two datasets are publicly available, while SM_ASL is a small-scale video dataset of ASL sign language that is created for this project. When building a sign language recognition system, feature extraction is usually the bottleneck for recognition delay. To mitigate this problem, the mediapipe library (mediapipe hand and mediapipe holistic) was used for feature extraction since it offers a lightweight machine-learning solution with fast inference even for devices with limited computation resources. Multilayer perceptron (MLP) and Long Short-term memory (LSTM) were used as the feature recognition model for static recognition and dynamic recognition respectively. Simulation results have shown that the static recognition pipeline was able to achieve an accuracy of 98% on the ASL alphabet. On the other hand, the dynamic recognition pipeline was able to achieve an accuracy of 94% on LSA64. The real-time recognition delay for both static and dynamic recognition is approximately 20 ms and 50 ms, respectively, making them suitable for real-time performance. While real-time static recognition on the ASL alphabet works smoothly without any issue, real-time dynamic recognition was only able to recognize 44/64 class correctly for LSA64. This is because there are many challenges in the training data of LSA64 such as color gloves and hand movement speed hindering hand tracking, and different video frames. To demonstrate the real-time performance of the dynamic recognition pipeline, the SM_ASL dataset was made which gives us complete control over the filming environment of training data and allows real-time dynamic recognition to be demonstrated smoothly.

Keywords: Sign Language Recognition, Computer Vision, Deep Learning

Table of Content

1. Introduction……..……..……6

2. Literature Review……..…..……8

3. Static Recognition……....……10

3.1. Dataset……...…..……10

3.1.1 Dataset augmentation……...…….11

3.2. Feature Extraction…….…….……..…11

3.2.1. Mediapipe hand……...…….11

3.2.2. Feature extraction with Mediapipe hand……...……. 12

3.2.3. Preprocessing……..…..…….. 12

3.3. Feature Recognition……...……..…13

3.3.1 Forward propagation……...…….…….13

3.3.2 Loss function……....…….15

3.3.3 Backpropagation……..…….16

3.4 Experimental Result……....……16

3.4.1. Experimental Setting……...……16

3.4.2. Simulation Result……...…..……17

3.4.3 Real-Time Recognition……...…….……18

4. Dynamic Recognition……..……20

4.1 Dataset…… 21

4.1.1 Data Augmentation…….…….…….21

4.2 Feature Extraction…….……..….21

4.2.1 Mediapipe holistic…….…….……..….22

4.2.2 Extraction process……...…...…..……22

4.2.3 Frame Equalizer…….…….……..…....…..23

4.2.4 Shift to center……...…….…….……..…23

4.3. Feature Recognition…….…….…..…24

4.4 Experimental Result…….…...…….…..…25

4.4.1 Experimental Setting……..…..……..…..…25

4.4.2 Simulation Result…….…….....…..…26

4.4.3 Real-Time Recognition…….…...…...…27

5. Conclusions and Future Works…….…….…...…28

1. Introduction

Sign language is a visual-based language used by the deaf community for communication within their community and also between those in the general population. It serves as a communication medium that relies solely on gestures, hand movements, and facial expressions without the need for hearing or speaking. Although it may look simple, sign language can be as complicated as spoken language. It has its own grammar and syntax and possesses the same level of expressive power as spoken language. Moreover, different countries often have their own variation of sign language. This added another layer of complexity which make it difficult for new people to learn sign language, and for existing users to communicate with another user from a different region.

The diversity of the existing sign language signifies their importance to communication for the deaf community; however, it also presents an obstacle for potential and current users. The major difficulty for sign language users is the amount of time and effort needed to learn new signs and their corresponding translations. One of the approaches to solving this problem can be found in the field of computer vision. By making use of automation to instantly recognize sign language into its corresponding textual or speech translation, it can help bridge the gap between sign language users and non-user, as well as between sign language users who are only accustomed to their own regional sign language. This subfield of research in computer vision is known as sign language recognition.

The importance of this research subfield boils down to its contribution to inclusive technology, which is a term used to describe any type of technology whose main purpose is to help those with disability work and function in society without being limited by their disability. A sign language recognition system can help aid communication between those with hearing impairment and those who do not know sign language. If such a system can be installed in, for example, a hospital, it can help deaf patients communicate with medical staff by translating sign language into spoken language. This would allow medical staff to understand the patient's needs better and provide the appropriate medical attention. The application of such a system extends beyond hospitality since it can help make any type of service more accessible to people with hearing impairment. If public facilities such as airports, train stations, education institutions, etc. were to be equipped with a sign

language recognition system, it can help provide real-time recognition of sign language which is essential for the communication and social inclusion of people with hearing disability.

Because of their application in helping people with hearing impairment, sign language recognition has received significant research attention in passing years. Past research in the area has achieved remarkable success by using deep learning approaches [1]. However, most of the proposed approaches either involve the use of peripheral hardware for data acquisition or deep learning models with high computational complexity for feature extraction and/or classification.

As such this project aims to tackle both of these issues by attempting to outline the methodology and build a sign language recognition system that employs lightweight models and requires no additional hardware. These key attributes allow the system to be deployed on mobile devices that have limited storage and computation power. Two recognition pipelines: static recognition and dynamic recognition will be explored. The methodology used for both pipelines is very similar. Both the static and dynamic recognition pipelines will make use of the Mediapipe framework for feature extraction, which provides a state-of-the-art lightweight machine-learning solution for hand posture and body posture recognition. This will eliminate the need for specialized hardware by replacing it with a feature extraction model with fast inference for real-time performance. Multilayer perceptron is the recognition model chosen for the static recognition pipeline since the extracted hand feature only contains spatial features. On the other hand, LSTM was used in the dynamic recognition pipeline because we need to take into account the temporal feature which is the changes in the spatial coordinate of the hand from frame to frame. Finally, the performance of each pipeline will be evaluated on the appropriate dataset: alphabet ASL for static recognition, and LSA64 and SM_ASL for dynamic recognition.

The rest of this paper will be structured as follow. Section 2 will present a literature review of different approaches for doing sign language recognition. Section 3 will outline the methodology this project used to do static recognition including the dataset, feature extraction method, preprocessing step, and hand sign recognition. Section 4 will describe the methodology for doing dynamic recognition. While section 4 will be very similar to section 3, some major preprocessing steps distinguish the two. At the end of sections 3 and section 4, simulation results and real-time recognition demos of each pipeline will be presented.

2. Literature Review

Current literature discussion on sign language recognition can be generally classified into two types: sensor-based method and vision-based method. Sensor-based approaches are often characterized by simpler recognition model; however, it requires the use of specialized hardware that are often uncommon. Some of this peripheral hardware include depth cameras like Microsoft Kinect, a leap motion camera, and a sensor glove. Each of them differs in the method of data acquisition, however, they all were designed to extract features from hand posture.

In [2], the author proposes a real-time hand gesture recognition system with image and video capture by Microsoft Kinect. Using a convolutional neural network trained on 90 000 RGB and depth images, they were able to achieve a predictive accuracy of 98.81% on 36 static gesture. Furthermore, the same data acquisition method was used for training convolutional LSTM and achieved a predictive accuracy of 99.08% for 10 dynamic hand gesture recognition in Indian sign language.

Another popular sensor-based method is to use a leap motion controller for data acquisition. In [3], the authors propose a sign language recognition system that uses a leap motion controller to capture a 3D infrared model of the human hand. For each frame, 32 features were extracted describing the position of the palm, wrist, elbow, direction of movement, and angles between specific joints. By making use of LSTM as a recognition model, they were able to achieve high accuracy on two small-scale Arabic sign language datasets. Their experiment shows an accuracy of 89% for single-handed signs with 13 classes and 96% for two-handed signs with 7 classes.

The sensor-based system in [4] used a customized glove-like wearable fitted with 6 inertial measurement units (IMU) to capture hand posture data. By introducing an IC2 multiplexer as an interface between the IMUs and a teensy 3.2 microcontroller (MCU), it allows the MCU to process the signal from all 6 sensors concurrently. Using these design, 156 features were extracted for each timestep and is used as input for an LSTM-based classification model. The proposed approach was able to recognize 28 word-based gestures with a 99.89% accuracy.

A sign language recognition system can also be built using a vision-based method. In a sensor-based method, the feature extraction stage is handled by the hardware. However, the vision-based method is characterized by the lack of a feature extraction stage (using raw RGB video as

input) or utilizing a model for feature extraction. Moreover, the recognition model used for classification tends to be more complex, and sometimes multiple models are used at the same time to achieve higher predictive accuracy. Even though it doesn't require specialized hardware, these characteristics introduce additional complexity to the system.

The authors in [5] propose a hand sign recognition system that makes use of single and parallel 3DCNN for classification. In the single 3DCNN architecture, 16 frames were linearly sampled from each video so that spatiotemporal features can be learned by the model. In the parallel 3DCNN, 32 frames were linearly sampled to be processed by 3 instances of 3DCNN. This approach enhances the contribution from temporal feature since each model process 16 frames with 50% overlapped. The two approach was evaluated on signer dependent KSU-SSL dataset and achieve an accuracy of 96.69% and 98.12% for single and parallel architecture respectively.

The sign language recognition in [6] was able to achieve an accuracy of 98% on the LSA64 dataset. It makes use of VGG19 to extract 18-body and 21-hand 2D joint coordinates. To introduce additional spatial features, the joint-line distance was computed between each joint and its projections on the lines formed by every other skeleton joint pair. The pipeline uses a combination of LSTM, LDS histogram, and meta-learning based on multi-layer perceptron for feature recognition. Despite its high accuracy on LSA64, both the feature extraction and feature recognition stages incur high computational complexity as a result of using multiple deep-learning models to make the prediction.

Both sensor-based and vision-based approaches offer different benefits and tradeoffs for building a sign language recognition system. In the sensor-based method, the feature extraction stage relies solely on hardware to generate input for the recognition model. While they offer a wide range of input data that is useful for feature recognition, this additional hardware is usually very expensive, while others are customized hardware that is hard to obtain. Moreover, hardware such as sensor gloves could also cause inconvenience to the user making the experience feel unnatural. Vision-based is an alternative approach for sign language recognition that doesn't require additional hardware. However, it either needs to use image or video as raw input, or use an additional model for feature extraction which adds computational complexity to the pipeline. The choice of each method would largely depend on the preference for user experience, the dataset size, and the available computational power.

3. Static Recognition

Static sign language recognition refers to the recognition of sign language on a single image or one frame from a video. The input is generally a still image of a person's hand that can convey meaning using only the hand posture. In other words, these hand signs represent their corresponding meaning by making use of the orientation of the hand such as the direction that the palm is facing (toward, away, left, or right of the body) and the configuration of the five fingers.

Static sign language usually has the following characteristic

- The translation must depend only on hand posture
- Performing the sign doesn't involve movement of the hand
- The position of the hand relative to the signer's body is irrelevant

While these characteristics contribute to the simplicity of static sign language, it also limits the expressive power these signs can have. Most static sign languages such as the American Sign Language (ASL) have only a few dozen classes that represent letters and numbers in the language. However, this simplicity also helps ease the development of a static sign language recognition system. Since the movement of the hand isn't involved, the system can perform recognition using only spatial features that can be obtained from a single frame of a video. Moreover, it can disregard any information about the signer's body posture since the correct translation will only depend on features extracted from a person's hand. As such it greatly simplifies the recognition model used in such a system. The training time required for the model will be shorter, and it will result in a low prediction delay when deploying the model for real-time recognition.

3.1 Dataset

The ASL dataset chosen for this project is an open-source dataset available on Kaggle [7]. In total, it consists of more than 223, 000 images belonging to 29 classes. The first 26 classes are letter-level hand signs for the English alphabet. The additional 3 classes correspond to "del", "nothing" and "space". Each image contains a static hand sign performed by a different person. Since each person has a similar but slightly different hand size, this variation eliminates the risk that the model will bias its prediction on hand size. The signs are also performed at a different

distance from the camera under different lighting and background. This ensures the model's robustness to variation in the hand distance from the camera.

3.1.1 Dataset augmentation

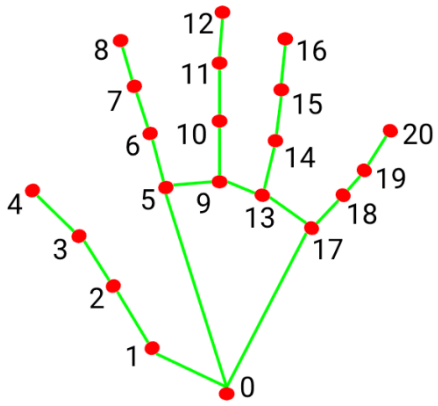
Most of the images in the dataset are hand signs perform using the right hand. Preliminary training on this dataset show that the model can recognize hand sign perform using the right hand correctly, however, it struggles to correctly classify the hand sign when it is performed using the left hand. There are several ways to solve this. One way is to distinguish between the left and right hand but this introduces additional features as input and thus making the model slightly more complex. Instead, we used data augmentation to solve this right-hand bias issue. Each image was augmented by randomly flipping it horizontally. As such about half of the original images will be flipped and serve as sample images for the left hand.

3.2 Feature Extraction

For feature extraction, the Mediapipe library was chosen since it can be readily integrated into the project. Mediapipe is a lightweight cross-platform machine-learning solution that offers fast inference even on common hardware. The system design for static recognition makes use of the Mediapipe hand for the extraction of the hand's landmark.

3.2.1. Mediapipe hand

The machine learning pipeline of Mediapipe consists of two models: a palm detector and a landmark extractor model. The palm detector output a bounding box on the full image around the detected hand(s), while the landmark extractor operates on the cropped region to infer 3D hand landmarks. The pipeline output 21 landmark each consisting of an (x, y, z) coordinate. The x and y coordinate describe the vertical and horizontal location of the landmark in the image and are normalized between 0 and 1 by the image height and width respectively, while the z coordinate describes the depth of the landmark with the origin near the wrist [8].



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

Figure 1: hand landmark location extracted by mediapipe hand

3.2.2. Feature extraction with Mediapipe hand

Since static recognition only required spatial features from the hand, we can directly use Mediapipe hand for feature extraction. For each image, 63 features are extracted and got appended to a CSV file. At the end of each line, a one-hot encoding of the corresponding class is appended. After the feature extraction process, the i image and its label can be mathematically defined as $X_i \in \mathbb{R}^{1\times 63}$and $y_i \in \mathbb{R}^{1\times 26}$

3.2.3. Preprocessing

Due to environmental factors such as lighting and bad hand position, mediapipe was not able to detect the hand in some images. These images were considered to be irrelevant and is discarded from the training data. To improve the accuracy and convergence time during training, each feature is normalized by its respective mean and standard deviation across all the samples based on equation (1).

$$X_i^j = \frac{X_i^j - mean(X^j)}{std(X^j)} \quad (1)$$

where:

X_i^j is the j feature of image i

X^j is a vector of j feature from all the image

3.3 Feature Recognition

The 33 coordinates extracted from each image are a spatial feature that describes the position and orientation of the hand present in the image. Since there is no temporal data to be considered, the recognition model only needs to learn a polynomial mapping between the input feature and its one hot encoding label.

$$Model(X^{1\times 63}) = y^{1\times 26} \quad (2)$$

With these assumptions, we follow the work in [9] and [10] which used multi-layer perceptron for hand sign recognition. This is the simplest form of a deep neural network, which consist of multiple layers of neuron fully connected with the layer adjacent to it.

Layer type	Output shape	activation
Input	63	None
Dense	32	Relu
Dense	32	Relu
Dense	26	Softmax

Table 1: model architecture for static recognition

As shown in Figure 2, the model has two hidden layers each output 32 hidden states. The relu activation function is used in the hidden layer while the final layer uses the SoftMax activation function with 26 outputs. The model is trained using categorical cross entropy as the loss function and the weights are optimized using the Adam optimizer with 10^{-3} as the learning rate.

3.3.1 Forward propagation

This section will formalize the forward pass equation for the model in Figure 1. These equations are what govern the calculation of the model's output and all intermediate states in between. Since the model has two hidden layers and one output layer, the forward pass will consist of 3 equations.

In order to simplify the formulation, let's assume that we are working with a feature vector $X^{1\times 63}$ representing one input example. However, the derivation can be directly applied to multiple input example.

Given an input vector $X^{1\times 63}$, the intermediate hidden state in an MLP can be calculated using Equation (3):

$$H = W_H X + b_H \tag{3}$$

Given: $H \in \mathbb{R}^{1\times h}$, $W_H \in \mathbb{R}^{h\times 63}$, and $b_h \in \mathbb{R}^{1\times h}$

Where h is the number of outputs of the hidden state

The ReLu activation function is defined as follows:

$$ReLu(x) = \begin{cases} 0, & x < 0 \\ x, & x > 0 \end{cases} \tag{4}$$

Since the Relu activation function is used in the hidden layer, equation (3) become

$$H = ReLu(W_H X + b_H) \tag{5}$$

The forward pass equation for the output layer is similar to equation (3), however, the SoftMax activation function is used instead of ReLu activation. Thus, the result from the output layer is given as:

$$O = Softmax(W_O H + b_O) \tag{6}$$

Given: $O \in \mathbb{R}^{1\times 26}$, $W_O \in \mathbb{R}^{26\times h}$, and $b_O \in \mathbb{R}^{1\times 26}$

Where: h is the number of outputs in the previously hidden layer.

Now that we have defined the forward propagation for each type layer, we can write the end-to-end forward pass equation as follows:

$$H_1 = ReLu(W_{H_1} X + b_{H_1}) \tag{7}$$

$$H_2 = ReLu(W_{H_2} H_1 + b_{H_2})$$

(8)

$$O = Softmax(W_O H_2 + b_O)$$

(9)

Since the SoftMax activation function is used in the output layer, the output can be thought of as a probability of the input belonging to a specific class. From Equation 9, we define

$$O' = W_O H_2 + b_O$$

(10)

The probability that a sample belongs to a class i as defined by the SoftMax function is

$$O_i = \frac{\exp\,(O_i')}{\exp\,(\sum_j O_j')}$$

(11)

Following from the property of probability, we also have the following equation

$$\sum_i O_i = 1$$

(12)

3.3.2 Loss function

The most commonly used function for a neural network with SoftMax output is the cross-entropy loss function because the loss value is directly proportional to the probability of the correct class. Given that, a vector X_k belong to class c, the general equation for calculating cross-entropy loss is given as follows:

$$L = -\sum_{k=1}^{N} P_{observe}(X_k) \cdot \log\,(P_{predicted}(X_k))$$

(13)

Where N is the number of training examples and $P_{predicted}(X_k)$ is the predicted probability X_k belong to class c. Moreover, $P_{observe}(X_k) = 1$ since we already know the correct label for X_k.

Therefore, the loss function can be simplified to:

$$L = -\sum_{k=1}^{N} \log\left(P_{predicted}(X_k)\right), \qquad where: P_{predicted}(X_k) = O_c \tag{14}$$

One distinguished property of cross entropy is that $\log\left(P_{predicted}(X_k)\right) = 0$, when $P_{predicted}(X_k) = 1$. That is when we are certain that X_k belong to class c, the loss value f disappears X_k disappear. Conversely, if the probability for class c is very low, $-\log\left(P_{predicted}(X_k)\right)$ will become very large, increasing the loss value.

3.3.3 Backpropagation

To optimize the neural network's weight, the Adam optimization algorithm [11] is used in backpropagation. Adam, originally proposed in 2014, is an algorithm for weight optimization with low time and space complexity. It updates the weight by approximating the 1st moment (mean) and 2nd moment using a moving average. The hyperparameter of Adam is intuitive to interpret since they represent the exponential decay rate of the 1st and 2nd moment estimate and the step size.

3.4 Experimental result

3.4.1 Experimental setting

The experiment was conducted in Python 3.9 on a laptop computer with 8GB of RAM and an 11th Gen Intel core i7 2.80 GHz processor. The input image for live recognition was captured using an integrated webcam with a resolution of 640 by 480 pixels. A sequential model was created using TensorFlow Keras consisting of 1 input layer, 2 hidden dense layers, and 1 output layer. Initially, two dropout layers were also added between the two hidden layers, but it was subsequently removed so that it has no effect on the final convergence result. The final model has 3962 trainable parameters.

3.4.2. Simulation result

The model was trained using an Adam optimizer with a learning rate of 10^{-3}. The moment exponential decay rate was chosen based on the recommendation by [11]. In total, there were 145,179 training samples. 75% was used for the training set. The remaining 15% and 10% were used for the test set. The model was trained for 100 epochs, while monitoring the cross-entropy validation loss at each epoch. At the end of every epoch, the checkpoint with the lowest validation loss so far will be saved for evaluation.

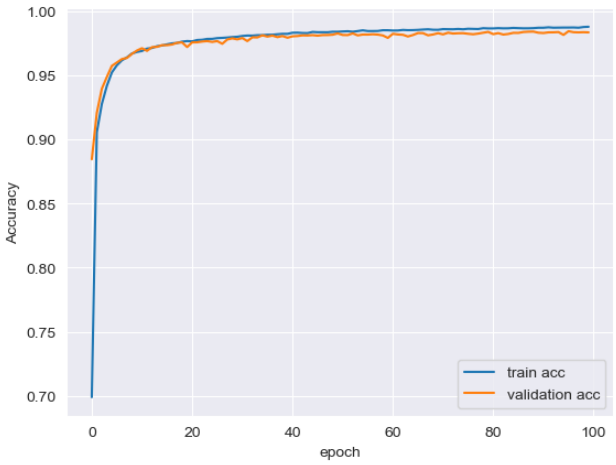


Figure 2: accuracy of training for static recognition

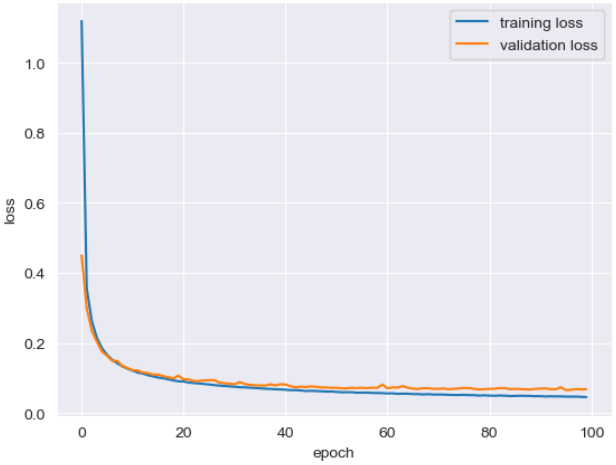


Figure 3: loss function for static recognition

Figure 3 and Figure 4 summarize the training process. After the 1st epoch, the model starts to converge to a validation accuracy of over 85%. This behavior causes the validation loss to be lower than the training in the beginning. This does not signify that the model is overfitting on the validation set. To verify this, a new model was initialized and evaluated on the validation set, which achieve an accuracy of only 4%. Therefore, we can conclude that the model learned the validation set quicker at earlier epochs. This behavior gradually disappears as the training progress. As the model approach epoch 20, we can see that the training loss starts to become smaller than the validation loss. This behavior persists until the end of epoch 100 when the training is stopped. It was able to achieve an accuracy of about 99% on the training set and 98% on the validation set. The final model was evaluated on a test set which achieve an accuracy of 98%.

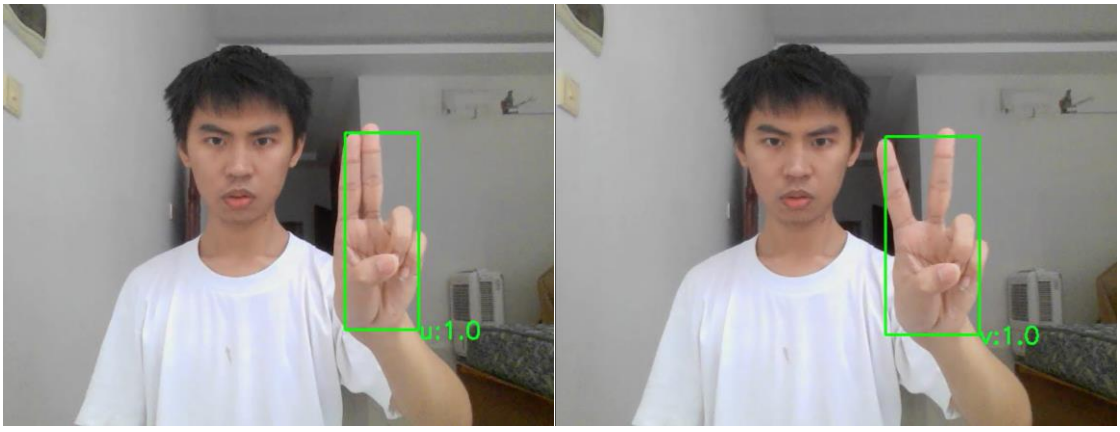
3.4.3 Real-recognition

After achieving a test set accuracy of at least 90%, the model was deployed to do recognition in real-time. It was able to recognize a majority of the 26 classes without any struggle. There are a few key notes that should be emphasized for this part of the experiment.

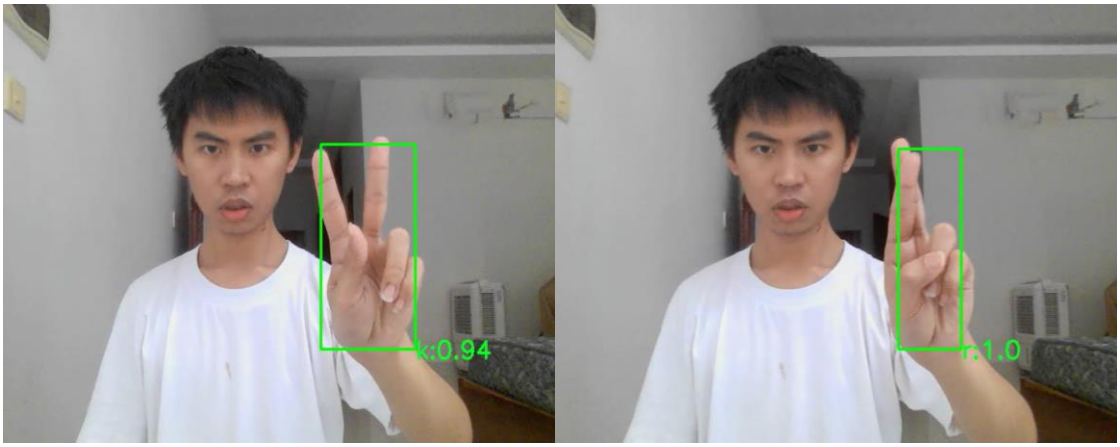
The time required to do prediction on each frame is approximately 20 ms, making it suitable for real-time recognition. The model was able to recognize hand signs perform by both the left and right hand. This is the result of dataset augmentation where the predominately right-hand dataset is randomly flipped on the horizontal axis. The characters u, ,v, ,r, and k are chosen since their hand posture greatly resemble each other. The model was able to correctly predict each class with over 95% confidence rate. The hand sign for the characters m and n are almost identical with the only subtle difference being the position of the thumb under the 4 fingers. Although the model struggled to distinguish these two hand signs, it was still able to recognize the characters correctly when the hand sign is presented at the right angle. Thus, we can conclude that the model is very sensitive to slight changes in hand posture when the hand sign is very similar.

(a) (b)

Figure 4: live demo of static recognition with both hand



(a) (b)



(c) (d)

Figure 5: live demo of 4 static signs: u, v, k, r

4. Dynamic Recognition

Dynamic recognition refers to the recognition of sign language based on a video or a sequence of frames. Unlike static recognition, the input to a dynamic recognition system will consist of a series of video frames representing the movement of the signer's hand relative to the body. Dynamic sign language is very versatile. By making use of both hand and body posture, dynamic sign language can be used to represent many words, and some commonly use sentences. While in most cases the body will remain stationary, the signer's hand can perform a wide range of motion throughout the frame. Both hands may be used to perform the sign, while each hand posture may gradually transition from one form to another.

In summary Dynamic sign language usually have the following characteristic:

- The translation depends on the hand posture and its movement
- It may involve one or both hand
- The position of the hand relative to the body is crucial to the translation.

Although these characteristics allow dynamic sign language to have greater expressive capability, it also introduces many challenges to the development of a dynamic sign language recognition system. The number of classes is generally very large since the translation are either words or sentences. The model for dynamic recognition is higher in complexity since both the hand and body posture are needed as input. Moreover, it also needs to take into account hand movement therefore temporal features will also need to be encoded into the input feature. Since we are working with video with a different number of frames, we will also need to decide how many frames to perform the recognition on. Too many frames may cause performance to slow, while using too few frames may leave out crucial spatial and temporal features needed for the recognition. These factors added another layer of complexity to the recognition model for dynamic signs. Not only must it take into account both spatial and temporal features, but it also needs to decide the irrelevant temporal feature that needs to be eliminated to improve performance. The recognition model takes relatively longer to train than static recognition, while the recognition delay during deployment is slightly higher.

4.1 Dataset

In order to benchmark the performance of the proposed dynamic recognition pipeline, the model was trained on the LSA64 dataset [12], [13]. it consists of 64 classes performed by 10 non-expert signers where each sign is repeated 5 times. In total, LSA64 consist of 3200 video which is relatively small in number therefore data augmentation is used to increase the sample size. The hand sign in LSA64 is subdivided into 42 one-handed signs and 22 two-handed signs. Some video was recorded in an outdoor environment with natural lighting while other were recorded indoors with artificial light. The signers were either sitting or standing while performing the sign. They wore black clothes with fluorescent gloves against a white background. The recording was done using a Sony HDR-CX240 camera mounted on a tripod 2 m away from the wall at a height of 1.5m. The temporal segmented version of LSA64 was used in this project. These sets contain videos that had been edited so that the first frame is the beginning of the sign execution while the last frame is the end of the sign execution. Each video has a resolution of 1920 x 1080 at 60 frames per second.

Another dataset that was used to train the model is a small-scale ASL dataset consisting of 8 classes with 30 videos per class. The video is recorded using an Android phone model OPPO F11 pro on the camera with a resolution of 1920 x 1080 at 30 frames per second. The phone is mounted on a tripod 1.5 m away from a wall. The subject performs the sign while sitting down. This allows us to control the dataset parameter such as the speed of hand movement, hand shape, and lighting which is crucial to the detection by Mediapipe.

4.1.1 Data augmentation

The sample size of LSA64 is relatively small compared to other dynamic sign language datasets. Moreover, all one-handed sign is performed using the right hand. Therefore, each video was flipped horizontally so that the total sample size is 6400 videos. This ensures that the model can recognize hands performed by both hands correctly while also increasing the number of data samples we can train on.

4.2 Feature extraction

Just like static recognition, the Mediapipe library is chosen for feature extraction. Since we also need pose landmarks, Mediapipe holistic was used instead of Mediapipe hand.

4.2.1 Mediapipe holistic

Mediapipe holistic [14] is an integrated solution for face [15], pose [16], and hand landmarks. The model output 468 landmarks for landmarks, 33 landmarks for body pose, and 2 x 21 landmarks for both hands. Each face and hand landmark consist of an (x, y, z) tuple where x and y denote the 2D position in the image and are normalized between 0 and 1 by the image width and height. The z coordinate represents how far the landmark is from the camera. Each pose landmark consists of a tuple (x, y, z, v) where the additional v represents the probability that the landmark is in the frame.

4.2.2 Extraction process

Only pose and hand landmark from Mediapipe is considered since the majority of the sign in the dataset only depend on hand movement relative to the body. Following the work in [17] and [18], which focus on recognition using 2D coordinates and eliminating the variation in z coordinate caused by camera distance, it was truncated from the extracted feature. With this assumption, we have (33 + 2x21) x 2 2D coordinates extracted by Mediapipe holistic from each frame of a video. Since some signs are performed using only one hand, the feature of the undetected hand is set to 0 by default.

Mediapipe Holistic struggles to extract hand landmarks from LSA64 because of fast hand movement and the fluorescent glove worn by the signer. In order to reduce the number of frames that Mediapipe cannot detect, we follow an approach mentioned in [19]. Each video was converted to grayscale before passing into Mediapipe. However, Mediapipe requires 3-channel input so the grayscale frame is stacked over each other 3 times to create a pseudo-3-channel frame.

For the small-scale ASL dataset, the feature can be extracted immediately by Mediapipe while filming the video. We shall call this dataset SM_ASL (Self-made ASL) dataset. Since we can control the number of frames, each video is filmed for 30 frames in order to minimize preprocessing step. Unlike LSA64, we can fine-tune the filming environment so that Mediapipe Holistic can detect hand landmarks in all frames of the video with minimal struggle. As such we also included the z coordinate as part of the input feature since the camera distance can be controlled.

4.2.3 Frame Equalizer

Since the video in LSA64 varied from 1 to 4 seconds, the number of frames in each video is not the same. If we do not enforce the number of frames used to do the recognition, it could impact the accuracy when training the model and can result in some inconvenience when deploying the model for real-time recognition. In order to solve this issue with LSA64, 30 frames were linearly sampled from each video in LSA64 using the Algorithm 1. The video from the SM_ASL dataset doesn't need this frame sampling since we already enforced that each video can only have 30 frames.

Algorithm 1: Linearly sample frame from each video

SampleFrame $(FrameList, numFrameOut)$;
Input : a list of all frames and a non-negative integer $numFrameOut$
Output: a list of size $numFrameOut$
$out \leftarrow List_{empty}$;
$L \leftarrow len(FrameList)$;
$i \leftarrow 1$;
while $i \neq numFrameOut + 1$ **do**
    $idx \leftarrow i/numFrameOut * L$;
    $out.append(frameList[idx])$;
    $i \leftarrow i + 1$;
end
return out;

4.2.4 Shifting to center

Another major issue with the extracted feature is that it varied depending on the position of the signer in the frame. If we train the model on the extracted feature directly, it will struggle to recognize the sign that is performed slightly outside the center of the frame. In order to make the model more robust to the signer's position, each of the extracted features is shifted to the center using the nose coordinate from the pose landmark as the center. After the feature extraction process, the i image and its label can be mathematically defined as $X_i \in \mathbb{R}^{f_n \times n_f}$ where $f_n = 30$ is the number of frame and n_f is the number of features extracted.

4.3 Feature recognition

Since our training data consist of n_f spatial features spanning f_n timesteps, the recognition model must have the ability to utilize the temporal feature across each time step so that it can achieve high predictive accuracy. Following the work in [20] and [21], LSTM is chosen as the recognition model.

Long short-term memory or LSTM is a recurrent neural network proposed in 1997 [22]. It aims to address the vanishing and exploding gradient problem when training a recurrent neural network on data with long-term temporal dependency. Unlike conventional RNN, LSTM makes use of a memory unit called gate to allow the gradient to influence the output over many time steps without vanishing or exploding. There are 3 input gates in LSTM: input gate, forget gate, and output gate. All three of these gates make use of the input and previous hidden state to update their gate value. Another component of LSTM is called the input node which makes use of a similar operation to the gate to update its value but it uses tanh activation instead of sigmoid.

The forward propagation of LSTM is given as follows:

$$f_t = Sigmoid\left(W_f x_t + U_f h_{t-1} + b_f\right)$$

$$i_t = Sigmoid(W_i x_t + U_i h_{t-1} + b_i)$$

$$o_t = Sigmoid(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot tanh(c_t)$$

(15)

As shown, the operation of the three gates and input node greatly resemble each other. Once these gate values are calculated, the memory cell's internal state is calculated using the input node, forget gate, and input gate. The input gate determines the relevant data in the input node while the forget gate decreases the influence of relevant data from the input node. Finally, the next hidden state is calculated using the output gate and the memory cell's internal state.

Layer type	Output shape	Activation
Input	$(f_n \ , \ n_f)$	None
LSTM	64	ReLu
LSTM	128	ReLu
LSTM	64	ReLu
Dense	64	ReLu
Dense	32	ReLu
Dense	Number of classes	SoftMax

Table 2: model architecture for dynamic recognition

The LSTM architecture in this project consist of 3 LSTM and 3 Dense layers. The pipeline consists of 5 hidden layers. The first 3 layers are LSTM modules with (64, 128, 64) units respectively. The first 2 layers must output a hidden state to be used in the next LSTM layer. The last 2 hidden layers consist of fully connected perceptron with (64, 32) units respectively. Relu is used as the activation function in all 5 hidden layers while the output layer uses the SoftMax activation function. The model is trained with cross entropy as the loss function and Adam optimizer with 10^{-5} as the learning rate. The detail for ReLu activation, Softmax, cross-entropy, and Adam optimizer is described in section 3.4

4.4 Experimental Result

4.4.1 Experimental Setting

The experimental setting for Dynamic Recognition is almost identical to that of static recognition as described in section 4.5.1 with a few slight adjustments. Instead of using the integrated webcam, an Android Phone OPPO F11 pro model was used for the camera. It is mounted on a tripod 1.5 m away from the wall with natural (daytime) or artificial lighting (nighttime). The video resolution is 1920 x 1080 pixels at 30 frames per second. A sequential model was built using Tensorflow Keras consisting of 1 input layer, 3 LSTM layers, 2 hidden dense layers, and 1 output layer. The total trainable parameter varies depending on the dataset because of the number of features extracted from each dataset.

4.4.2 Simulation Result

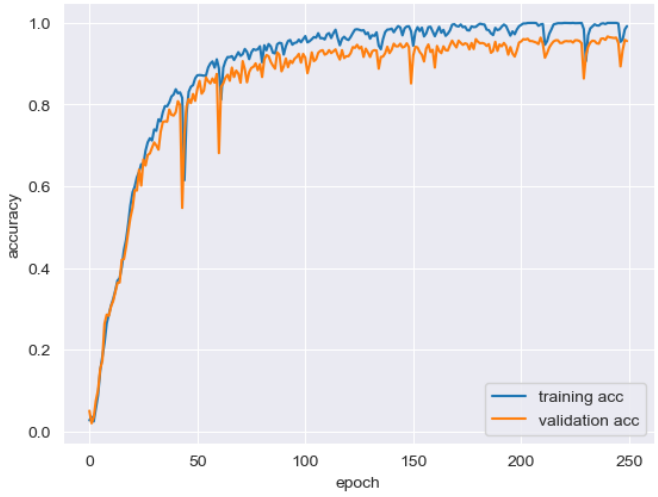


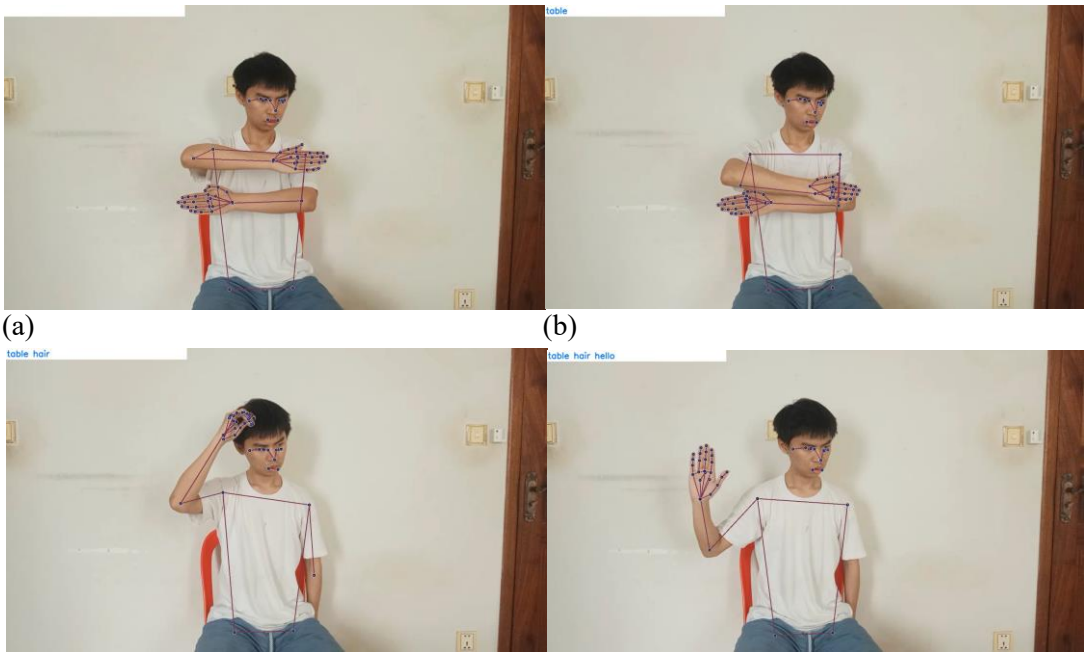
Figure 6: training accuracy for dynamic recognition

Figure 7: Training loss value for dynamic recognition

Figure 6 and Figure 7 illustrate the training process on LSA64. The model was trained for 250 epochs before training was halted. There were a few sharp spikes in the loss value such as those near epoch 50, however, the model eventually converges at the end of training. We were able to achieve 98% accuracy on the training and 95% accuracy on the validation. At the end of epoch 250, the final model is evaluated on a test set (10% of dataset size) and achieves an accuracy of 94%. Unlike in static recognition, the model seems to fit both the training and validation set at roughly the same rate in early epochs as seen by the overlapping of loss value in Figure 7.

4.4.3 Real-Time Recognition

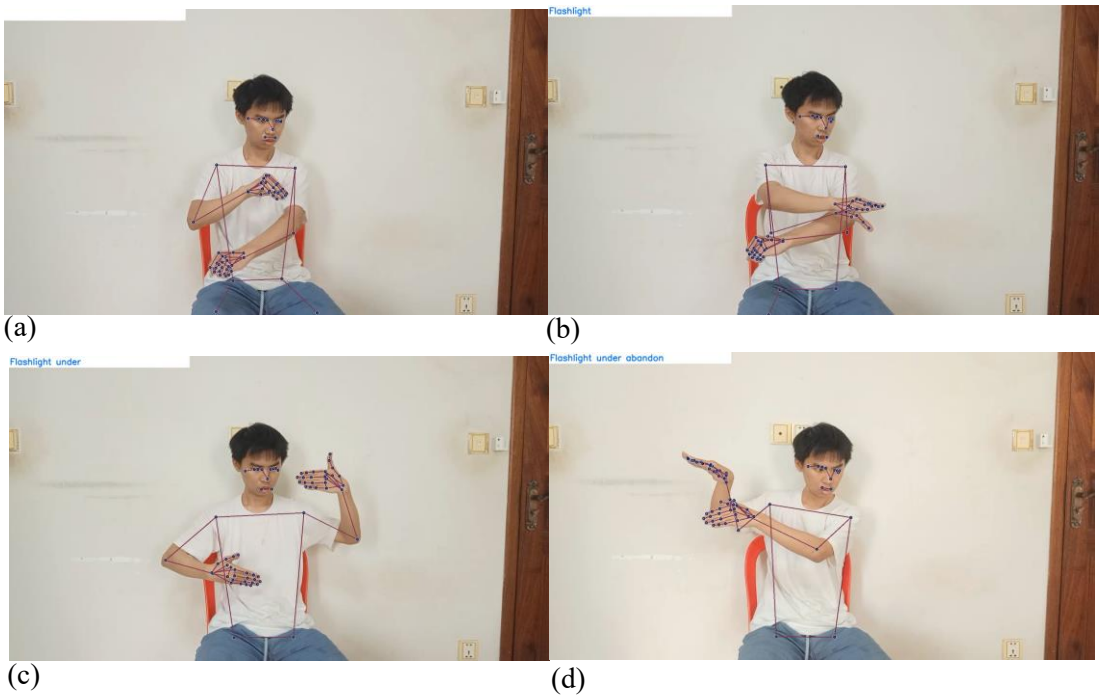
This section will discuss the result of real-time recognition for both LSA64 and the small-scale dataset. For LSA64, 44 out of the 64 classes were able to be recognized correctly in real time without any struggle. However, there were 22 classes that weren't able to be recognized without frequent adjustment of the camera angle. Moreover, Mediapipe Holistic sometime struggles to detect hands in the training data because of the hand speed and color glove. These factors cause the real-time recognition on LSA64 to be very unstable. In an effort to showcase the feasibility of the proposed dynamic sign language recognition pipeline, a small-scale dynamic hand sign dataset with 8 classes, called SM_ASL, was made. By having complete control over the camera distance, lighting, and hand movement, the real-time recognition of this dataset can be smoothly demonstrated.



(a) (b)

(c) (d)

Figure 8: live demo of 3 dynamic signs: table, hair, hello



(a) (b)

(c) (d)

Figure 9: live demo of another 3 dynamic signs: flashlight, under, abandon

5. Conclusions and Future Work

This section shall conclude the whole project with a brief summary and outline the potential future work that could add more value to this project. The static recognition pipeline make use of Mediapipe hand for feature extraction and MLP for feature recognition was able to achieve an accuracy of 98% on the ASL alphabet. On the other hand, the dynamic recognition pipeline making use of Mediapipe holistic for feature extraction and LSTM for feature recognition was able to achieve an accuracy of 94% on LSA64. Real-time static recognition can recognize all 26 letters of the English alphabet without any issue. However, we are only able to recognize 44/64 classes from LSA64 for real-time dynamic recognition. By training LSTM on SM_ASL we have complete control over the filming environment of the training data, and were able to achieve real-time dynamic recognition smoothly.

One of the major difficulties throughout this project is achieving good accuracy when doing real-time dynamic recognition. As aforementioned, this limitation stems from using Mediapipe for feature extraction. Although the mediapipe framework offers a fast and readily available solution for extracting hand features, it was only trained on bare hands and struggled to track hands with irregular shapes or fast movements. As such the video dataset need to be made specifically for

mediapipe so that it will be able to extract all frame in the video leaving no gap in the training data. Another advantage of making the dataset ourselves is that it allows us to restrict the number of frames each video can have. With these conditions, we can do data augmentation by generating new feature through Gaussian Distribution where the mean and standard deviation is defined below.

$$X_{aug}^{ij} \sim N(\ mean(X^{ij}),\ \ std(X^{ij})\ ) \tag{16}$$

$$where: mean(X^{ij})\ is\ the\ mean\ of\ i\ feature\ in\ the\ j\ frame\ of\ all\ video\ in\ class\ c$$

$$std(X^{ij})\ is\ the\ standard\ deviation\ of\ i\ feature\ in\ the\ j\ frame\ of\ all\ video\ in\ class\ c$$

By using equation 16, a new data sample of the same class can be generated given that we already have sufficient sample data for that class.

Reference

[1] M. Al-Qurishi, T. Khalid, and R. Souissi, “Deep Learning for Sign Language Recognition: Current Techniques, Benchmarks, and Open Issues,” *IEEE Access*, vol. 9, pp. 126917–126951, 2021, doi: 10.1109/ACCESS.2021.3110912.

[2] N. K. Bhagat, Y. Vishnusai, and G. N. Rathna, “Indian Sign Language Gesture Recognition using Image Processing and Deep Learning,” in *2019 Digital Image Computing: Techniques and Applications (DICTA)*, Perth, Australia: IEEE, Dec. 2019, pp. 1–8. doi: 10.1109/DICTA47822.2019.8945850.

[3] “Enhancing The Recognition Of Arabic Sign Language By Using Deep Learning And Leap Motion Controller”, [Online]. Available: https://www.researchgate.net/profile/Ammar-Alnahhas/publication/341218971_Enhancing_The_Recognition_Of_Arabic_Sign_Language_By_Using_Deep_Learning_And_Leap_Motion_Controller/links/5eb87f51299bf1287f7c91dd/Enhancing-The-Recognition-Of-Arabic-Sign-Language-By-Using-Deep-Learning-And-Leap-Motion-Controller.pdf

[4] T.-W. Chong and B.-J. Kim, “American Sign Language Recognition System Using Wearable Sensors with Deep Learning Approach,” *The Journal of the Korea institute of electronic communication sciences*, vol. 15, no. 2, pp. 291–298, Apr. 2020, doi: 10.13067/JKIECS.2020.15.2.291.

[5] M. Al-Hammadi, G. Muhammad, W. Abdul, M. Alsulaiman, M. A. Bencherif, and M. A. Mekhtiche, “Hand Gesture Recognition for Sign Language Using 3DCNN,” *IEEE Access*, vol. 8, pp. 79491–79509, 2020, doi: 10.1109/ACCESS.2020.2990434.

[6] D. Konstantinidis, K. Dimitropoulos, and P. Daras, “SIGN LANGUAGE RECOGNITION BASED ON HAND AND BODY SKELETAL DATA,” in *2018 - 3DTV-Conference: The True Vision - Capture, Transmission and Display of 3D Video (3DTV-CON)*, Helsinki: IEEE, Jun. 2018, pp. 1–4. doi: 10.1109/3DTV.2018.8478467.

[7] D. S. SAU, “ASL(American Sign Language) Alphabet Dataset.” https://www.kaggle.com/datasets/debashishsau/aslamerican-sign-language-aplhabet-dataset?resource=download, 2021.

[8] Google, “Mediapipe Hand.” https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

[9] D. Naglot and M. Kulkarni, “Real time sign language recognition using the leap motion controller,” in *2016 International Conference on Inventive Computation Technologies (ICICT)*, Coimbatore, India: IEEE, Aug. 2016, pp. 1–5. doi: 10.1109/INVENTIVE.2016.7830097.

[10] M. Mohandes, S. Aliyu, and M. Deriche, “Arabic sign language recognition using the leap motion controller,” in *2014 IEEE 23rd International Symposium on Industrial Electronics (ISIE)*, Istanbul, Turkey: IEEE, Jun. 2014, pp. 960–965. doi: 10.1109/ISIE.2014.6864742.

[11] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization.” arXiv, Jan. 29, 2017. Accessed: May 05, 2023. [Online]. Available: http://arxiv.org/abs/1412.6980

[12] F. Ronchetti, F. Quiroga, C. Estrebou, L. Lanzarini, and A. Rosete, “LSA64: A Dataset for Argentinian Sign Language,” 2016, [Online]. Available: http://sedici.unlp.edu.ar/bitstream/handle/10915/56764/Documento_completo.pdf-PDFA.pdf?sequence=1&isAllowed=y

[13] F. Quiroga, “LSA64: An Argentinian Sign Language Dataset.” 2016. [Online]. Available: http://sedici.unlp.edu.ar/bitstream/handle/10915/56764/Documento_completo.pdf-PDFA.pdf?sequence=1&isAllowed=y

[14] Google, “MediaPipe Holistic.” https://developers.google.com/mediapipe/solutions/vision/holistic_landmarker

[15] Google, “Mediapipe Face,” *mediapipe face*. https://developers.google.com/mediapipe/solutions/vision/face_landmarker

[16] Google, “Mediapipe Pose,” *mediapipe pose*. https://developers.google.com/mediapipe/solutions/vision/pose_landmarker

[17] M. Bohacek and M. Hruz, “Sign Pose-based Transformer for Word-level Sign Language Recognition,” in *2022 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, Waikoloa, HI, USA: IEEE, Jan. 2022, pp. 182–191. doi: 10.1109/WACVW54805.2022.00024.

[18] M. A. Bencherif *et al.*, “Arabic Sign Language Recognition System Using 2D Hands and Body Skeleton Data,” *IEEE Access*, vol. 9, pp. 59612–59627, 2021, doi: 10.1109/ACCESS.2021.3069714.

[19] I. Rodriguez-Moreno, J. M. Martinez-Otzeta, I. Goienetxea, I. Rodriguez, and B. Sierra, “A New Approach for Video Action Recognition: CSP-Based Filtering for Video to Image Transformation,” *IEEE Access*, vol. 9, pp. 139946–139957, 2021, doi: 10.1109/ACCESS.2021.3118829.

[20] E. Abraham, A. Nayak, and A. Iqbal, “Real-Time Translation of Indian Sign Language using LSTM,” in *2019 Global Conference for Advancement in Technology (GCAT)*, BANGALURU, India: IEEE, Oct. 2019, pp. 1–5. doi: 10.1109/GCAT47503.2019.8978343.

[21] T. Liu, W. Zhou, and H. Li, “Sign language recognition with long short-term memory,” in *2016 IEEE International Conference on Image Processing (ICIP)*, Phoenix, AZ, USA: IEEE, Sep. 2016, pp. 2871–2875. doi: 10.1109/ICIP.2016.7532884.

[22] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.